

การจัดการอัปโหลดไฟล์ใน **Node.js**

ด้วย **Multer, fs** และ **path**

ระดับใช้งานจริง Production

จัดทำโดย: Jirayu Suriwan

สำหรับนักพัฒนา Backend ระดับ Junior → Intermediate

ปัญหาที่พบบ่อยในการอัปโหลดไฟล์

สิ่งที่มักเกิดขึ้นเมื่อไม่มีระบบจัดการที่ดี

อัปโหลดแล้วไฟล์หาย

ไม่ได้กำหนด destination ที่ถูกต้อง

ไฟล์ถูกเขียนทับ (Overwrite)

ชื่อไฟล์ซ้ำกัน ไม่มีระบบตั้งชื่อที่ดี

Path ผิดพลาด

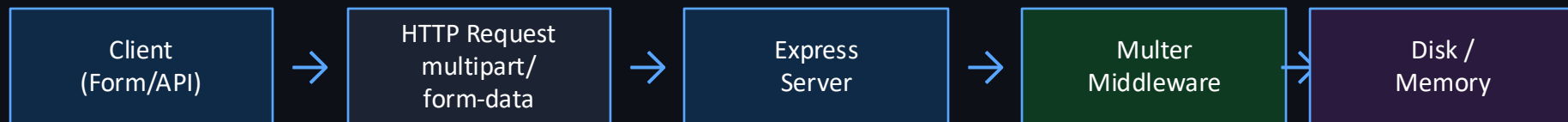
ใช้ hardcode path ทำให้ผิด OS

ไม่มี Validation

รับไฟล์ทุกชนิด/ทุกขนาด เสี่ยงโดน attack

ภาพรวมการอัปโหลดไฟล์

Flow: Client → Server → Multer → Storage | ใช้ multipart/form-data



multipart/form-data

- encoding type สำหรับส่งไฟล์ผ่าน HTTP
- แยก text fields และ file fields
- Browser จัดการ encoding อัตโนมัติ

Multer ทำอะไร?

- Parse multipart request
- แยกไฟล์ออกจาก body
- เก็บไฟล์ตาม config (disk/memory)
- เพิ่ม req.file / req.files

เทคโนโลยีที่ใช้ในบทเรียนนี้

Node.js

JavaScript Runtime สำหรับ Backend

Express.js

Web Framework ยอดนิยม

Multer

Middleware จัดการ File Upload

fs (File System)

Module จัดการไฟล์ของ Node.js

path

Module จัดการ path อย่างปลอดภัย

การเตรียมโปรเจกต์

Setup เบื้องต้นก่อนเริ่มพัฒนาระบบ upload

```
# สร้างโปรเจกต์ใหม่
npm init -y

# ติดตั้ง dependencies
npm install express multer
```

- แยก config ออกจาก route เพื่อ reusability
- สร้างโฟลเดอร์ uploads/ ไว้ล่วงหน้า
- ใช้ .gitignore เพื่อไม่ commit ไฟล์ upload

โครงสร้างไฟล์เบื้องต้น

```
project/
├── server.js
├── routes/
│   └── upload.js
├── middleware/
│   └── multerConfig.js
├── uploads/
│   └── (uploaded files)
└── package.json
```

Multer คืออะไร?

Multer คือ Node.js middleware สำหรับจัดการ `multipart/form-data` ซึ่งใช้สำหรับการอัปโหลดไฟล์เป็นหลัก

- ทำงานร่วมกับ Express.js ได้ทันที
- รองรับอัปโหลดไฟล์เดี่ยวและหลายไฟล์
- เลือกเก็บลง Disk หรือ Memory ได้
- มีระบบ filter ตรวจสอบชนิดไฟล์
- เป็น open source (MIT License) — นิยมใช้ในองค์กร

Multer ทำงานอย่างไร?

กระบวนการภายในเมื่อ request เข้ามา

1. Intercept Request

Multer ดักจับ request ที่เป็น multipart



2. Parse Form Data

แยก text fields ออกจาก file fields



3. Process File

ตรวจสอบ filter, ขนาด, จำนวนไฟล์



4. Store File

บันทึกลง Disk หรือเก็บใน Memory Buffer



5. Attach to req

เพิ่ม req.file / req.files ให้ใช้งานต่อ

การตั้งค่า Storage

diskStorage — กำหนด destination และ filename

```
const multer = require('multer');

// กำหนดที่เก็บไฟล์และชื่อไฟล์
const storage = multer.diskStorage({
  // กำหนดโฟลเดอร์ปลายทาง
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  // ตั้งชื่อไฟล์ไม่ให้ซ้ำ (ใช้ timestamp)
  filename: (req, file, cb) => {
    const uniqueName = Date.now() + '-' +
      file.originalname;
    cb(null, uniqueName);
  }
});
```

diskStorage Options

- destination — โฟลเดอร์ที่จะเก็บไฟล์
- filename — กำหนดชื่อไฟล์ที่บันทึก

แนะนำการตั้งชื่อ

- timestamp + originalname
- UUID v4 + extension
- hash + extension

ตัวอย่างโค้ด Multer พื้นฐาน

Express + Multer Setup ที่ใช้งานได้ทันที

```
const express = require('express');
const multer = require('multer');
const path = require('path');

const app = express();

// ตั้งค่า storage
const storage = multer.diskStorage({
  destination: 'uploads/',
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, Date.now() + ext);
  }
});

// สร้าง middleware
const upload = multer({ storage });

// API endpoint สำหรับอัปโหลด
app.post('/upload', upload.single('file'),
  (req, res) => {
    res.json({ file: req.file });
  });
```

อัปโหลดไฟล์เดี่ยว — `upload.single()`

```
// อัปโหลดไฟล์เดี่ยว — field name = 'file'
app.post('/upload',
  upload.single("file"),
  (req, res) => {
    console.log(req.file);
    // req.file มีข้อมูล:
    // filename, originalname, encoding,
    // mimetype, destination, filename,
    // path, size
    res.json({
      message: 'อัปโหลดสำเร็จ',
      filename: req.file.filename
    });
  });
```

`req.file` properties

- `fieldname` — ชื่อ field จาก form
- `originalname` — ชื่อไฟล์ต้นฉบับ
- `mimetype` — ชนิดไฟล์ เช่น `image/png`
- `size` — ขนาดไฟล์ (bytes)
- `path` — path เต็มของไฟล์ที่บันทึก

อัปโหลดหลายไฟล์ — `upload.array()`

```
// อัปโหลดหลายไฟล์ — จำกัดสูงสุด 5 ไฟล์
app.post('/upload-multiple',
  upload.array("files", 5),
  (req, res) => {
    console.log(req.files); // Array ของไฟล์
    const fileNames = req.files.map(
      f => f.filename
    );
    res.json({
      message: `อัปโหลด ${req.files.length} ไฟล์`,
      files: fileNames
    });
  });
```

methods อื่นที่น่าสนใจ

- `upload.single("file")` — ไฟล์เดียว
- `upload.array("files", max)` — หลายไฟล์
- `upload.fields([...])` — หลาย field
- `upload.none()` — ไม่รับไฟล์ (form only)

การตั้งชื่อไฟล์ — สำคัญมาก!

ป้องกันไฟล์ซ้ำ และ path injection

Timestamp

- `Date.now()` + extension
- ง่าย, เร็ว, ไม่ต้องติดตั้งเพิ่ม
- อาจชนกันถ้า upload พร้อมกัน

UUID v4

- `uuid.v4()` + extension
- ไม่มีทางซ้ำ (virtually unique)
- ต้อง `npm install uuid` เพิ่ม

```
const { v4: uuidv4 } = require('uuid');

// วิธีที่ 1: Timestamp
const name1 = Date.now() + '-' + file.originalname;

// วิธีที่ 2: UUID (แนะนำสำหรับ Production)
const ext = path.extname(file.originalname);
const name2 = uuidv4() + ext;
```

การใช้งาน fs (File System)

Module สำหรับจัดการไฟล์บน Server

```
const fs = require('fs');

// ลบไฟล์ (Async)
fs.unlink('uploads/old-file.jpg', (err) => {
  if (err) console.error('ลบไฟล์ไม่สำเร็จ:', err);
  else console.log('ลบไฟล์สำเร็จ');
});

// ตรวจสอบว่าไฟล์มีอยู่หรือไม่
if (fs.existsSync('uploads/photo.jpg')) {
  console.log('ไฟล์มีอยู่');
}

// ย้ายไฟล์ (rename)
fs.rename('temp/file.jpg', 'uploads/file.jpg',
  (err) => { if (err) throw err; });
```

ตัวอย่าง API ลบไฟล์

DELETE /api/file/:filename

```
const fs = require('fs');
const path = require('path');

// API สำหรับลบไฟล์
app.delete('/api/file/:filename',
  (req, res) => {
    const filePath = path.join(
      __dirname, 'uploads', req.params.filename
    );

    // ตรวจสอบว่าไฟล์มีอยู่ก่อนลบ
    if (!fs.existsSync(filePath)) {
      return res.status(404).json({
        error: 'ไม่พบไฟล์' });
    }

    fs.unlink(filePath, (err) => {
      if (err) return res.status(500).json({
        error: 'ลบไฟล์ไม่สำเร็จ' });
      res.json({ message: 'ลบสำเร็จ' });
    });
  });
```

การใช้งาน path Module

จัดการ path อย่างปลอดภัย ข้าม OS ได้

```
const path = require('path');

// path.join() - รวม path อย่างปลอดภัย
const filePath = path.join(__dirname, 'uploads', 'photo.jpg');
// Windows: C:\project\uploads\photo.jpg
// Linux:    /home/user/project/uploads/photo.jpg

// path.extname() - ดึงนามสกุลไฟล์
path.extname('photo.jpg'); // '.jpg'
path.extname('doc.tar.gz'); // '.gz'

// path.basename() - ดึงชื่อไฟล์
path.basename('/uploads/photo.jpg'); // 'photo.jpg'

// path.resolve() - สร้าง absolute path
path.resolve('uploads', 'photo.jpg');
```

ความเสี่ยงด้าน Security

สิ่งที่ต้องระวังในระบบ upload

Path Traversal (../)

ผู้โจมตีใส่ ../../etc/passwd ในชื่อไฟล์
เพื่อเข้าถึงไฟล์ระบบ

✓ ใช้ `path.basename()` ตัด path ออก

Upload ไฟล์อันตราย

อัปโหลดไฟล์ .exe, .sh หรือ malware
ปลอม MIME type เป็น image

✓ ตรวจสอบ MIME type + extension + magic bytes

File Overwrite

ไฟล์ใหม่เขียนทับไฟล์เก่า
ข้อมูลสูญหาย

✓ ใช้ UUID/timestamp ในชื่อไฟล์

การตรวจสอบไฟล์ (Validation)

fileFilter — ตรวจสอบก่อนรับไฟล์

```
// fileFilter — ตรวจสอบชนิดและขนาดไฟล์
const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
  fileFilter: (req, file, cb) => {
    // กำหนด MIME type ที่อนุญาต
    const allowed = [
      'image/jpeg', 'image/png', 'image/gif'
    ];
    if (allowed.includes(file.mimetype)) {
      cb(null, true); // อนุญาต
    } else {
      cb(new Error('ชนิดไฟล์ไม่ถูกต้อง'));
    }
  }
});
```

Validation Flow



X ปฏิเสธไฟล์

การจัดการ Error

Multer Error + Custom Error Handler

```
const multer = require('multer');

// Middleware จัดการ error จาก Multer
app.post('/upload', (req, res, next) => {
  upload.single('file')(req, res, (err) => {
    if (err instanceof multer.MulterError) {
      // Error จาก Multer เอง
      if (err.code === 'LIMIT_FILE_SIZE') {
        return res.status(413).json({
          error: 'ไฟล์ใหญ่เกินไป' });
      }
      return res.status(400).json({
        error: err.message });
    } else if (err) {
      // Error จาก fileFilter หรืออื่นๆ
      return res.status(400).json({
        error: err.message });
    }
    // อัปโหลดสำเร็จ
    res.json({ file: req.file });
  });
});
```

Performance และ Limit

ตั้งค่า limits เพื่อป้องกันระบบล่ม

```
const upload = multer({
  storage: storage,
  limits: {
    fileSize: 10 * 1024 * 1024, // 10MB ต่อไฟล์
    files: 5, // สูงสุด 5 ไฟล์
    fields: 20, // สูงสุด 20 fields
    fileSize: 1024 * 100, // field ข้อความ 100KB
  },
});
```

Memory Storage

- เก็บไฟล์ใน RAM (Buffer)
- เร็วมาก แต่ใช้ RAM สูง
- เหมาะกับไฟล์เล็ก / ประมวลผลก่อนเก็บ

Disk Storage

- เขียนลง disk โดยตรง
- รองรับไฟล์ใหญ่ได้ดี
- แนะนำสำหรับ Production

โครงสร้างโฟลเดอร์ที่แนะนำ

แยกส่วนให้ชัดเจน — ง่ายต่อการ maintain

```
project/
├── src/
│   ├── routes/
│   │   └── upload.routes.js
│   ├── middleware/
│   │   └── upload.middleware.js
│   ├── controllers/
│   │   └── upload.controller.js
│   └── utils/
│       └── fileHelper.js
├── uploads/           ← ไฟล์ถาวร
├── temp/              ← ไฟล์ชั่วคราว
├── .gitignore
└── server.js
```

/uploads

- เก็บไฟล์ที่ผ่าน validation แล้ว
- อาจแบ่ง subfolder ตาม type
- เช่น /uploads/images, /uploads/docs

/temp

- เก็บไฟล์ชั่วคราวก่อนประมวลผล
- ลบอัตโนมัติด้วย cron job
- ไม่ควร serve ให้ client โดยตรง

.gitignore: uploads/ และ temp/

การเปิดให้เข้าถึงไฟล์ (Static Files)

express.static() — mapping URL กับ folder

```
// เปิดให้เข้าถึงไฟล์ใน uploads/ ผ่าน URL
app.use('/files',
  express.static(
    path.join(__dirname, 'uploads')
  )
);

// เข้าถึงได้ที่:
// http://localhost:3000/files/1713945600000.jpg
```

- ใช้ /files แทน /uploads เพื่อซ่อน path จริง
- ตั้ง CORS ถ้า frontend อยู่คนละ domain
- พิจารณาเพิ่ม authentication ก่อนเข้าถึงไฟล์ส่วนตัว
- ใช้ CDN สำหรับไฟล์ static ใน Production

การใช้งานจริง (Production)

Best Practices สำหรับระบบองค์กร

ไม่เก็บไฟล์ใน **project root**

ใช้ path แยกที่ mount volume ได้

แยก **Environment**

dev ใช้ local disk, prod ใช้ Cloud Storage

ใช้ **Cloud Storage**

Amazon S3, Google Cloud Storage, Azure Blob

Logging & Monitoring

บันทึกทุก upload/delete + alert เมื่อ disk เต็ม

Backup Strategy

กำหนดนโยบาย backup ไฟล์สำคัญ

การลบไฟล์ที่ไม่ใช้ (Cleanup)

ป้องกัน disk เต็ม — ใช้ cron job ลบไฟล์ temp

```
const fs = require('fs');
const path = require('path');

// ฟังก์ชันลบไฟล์เก่าใน temp/ (อายุเกิน 24 ชม.)
function cleanupTempFiles() {
  const tempDir = path.join(__dirname, 'temp');
  const maxAge = 24 * 60 * 60 * 1000; // 24 hrs
  const now = Date.now();

  fs.readdirSync(tempDir).forEach(file => {
    const filePath = path.join(tempDir, file);
    const stats = fs.statSync(filePath);
    if (now - stats.mtimeMs > maxAge) {
      fs.unlinkSync(filePath);
      console.log("ลบ:", file);
    }
  });
}

// เรียกทุก 1 ชั่วโมง
setInterval(cleanupTempFiles, 60*60*1000);
```

ตัวอย่างจริง: Upload รูปโปรไฟล์

อัปโหลดรูปใหม่ → ลบรูปเก่า อัตโนมัติ

```
// อัปโหลดรูปโปรไฟล์ + ลบรูปเก่า
app.put('/api/user/avatar',
  upload.single('avatar'),
  async (req, res) => {
    try {
      const user = await User.findById(
        req.userId);

      // ลบรูปเก่า (ถ้ามี)
      if (user.avatar) {
        const oldPath = path.join(
          __dirname, 'uploads', user.avatar);
        if (fs.existsSync(oldPath)) {
          fs.unlinkSync(oldPath);
        }
      }

      // อัปเดตชื่อไฟล์ใหม่
      user.avatar = req.file.filename;
      await user.save();
      res.json({ avatar: user.avatar });
    } catch (err) {
      res.status(500).json({ error: err });
    }
  });
```


ข้อผิดพลาดที่พบบ่อย

หลีกเลี่ยงสิ่งเหล่านี้ในโค้ดของคุณ

1	ไม่ validate ชนิด/ขนาดไฟล์	→ ใช้ <code>fileFilter</code> + <code>limits</code> ทุกครั้ง
2	Hardcode path เช่น <code>'C:\\uploads'</code>	→ ใช้ <code>path.join()</code> + <code>__dirname</code>
3	ไม่ handle error จาก Multer	→ ใช้ <code>error middleware</code> ครอบ
4	ใช้ชื่อไฟล์ original โดยตรง	→ ตั้งชื่อใหม่ด้วย <code>UUID/timestamp</code>
5	ไม่ลบไฟล์ temp / ไฟล์เก่า	→ ใช้ <code>cleanup function</code> + <code>cron</code>
6	เก็บไฟล์ใน project root	→ แยก folder + ใช้ <code>Cloud Storage</code>

สรุป

เครื่องมือหลักสำหรับระบบ File Upload ใน Node.js

Multer

= Upload

Middleware จัดการอัปโหลดไฟล์
รองรับ single, array, fields

fs

= จัดการไฟล์

ลบ / ย้าย / ตรวจสอบไฟล์บน Server
unlink, existsSync, rename

path

= จัดการ Path

สร้าง path ที่ปลอดภัยข้าม OS
join, extname, basename, resolve

Key Takeaway: ออกแบบระบบ upload ให้ปลอดภัย + validate ทุกจุด + ใช้ Cloud Storage สำหรับ Production

Q & A

คำถาม & คำตอบ

ขอบคุณที่รับฟัง!